SIMT-Aware Lockstep Verification and Functional-Coverage Closure Methodology for an Open-Source RISC-V GPGPU: A UVM 1.2 Environment

***Abstract*—Open-source RISC-V GPGPUs such as Vortex ship with directed-kernel regressions but no industrial-grade verification: no reference-model checking, no functional-coverage model, and no sign-off discipline. This paper presents a complete UVM 1.2 environment and methodology that closes that gap. The environment wraps a bus-master SIMT DUT with role-inverted agents, integrates Vortex's functional simulator as a per-configuration golden model over DPI-C, and renders verdicts through two qualified checkers: a bidirectional end-state scoreboard and a per-instruction, per-lane lockstep comparator governed by five SIMT-specific alignment rules. A sound two-pass load-value feed makes racy multi-core programs instruction-granularity verifiable (residual zero over 5,432 retirements), with a characterized soundness boundary at asynchronous interrupt timing. A three-layer coverage model adds, to our knowledge, the first published SIMT functional-coverage layer for RTL GPU verification—warp divergence depth, scratchpad bank conflicts, and memory-coalescing classes—closing the environment's own layers at 98.1% covergroup-bin / 94.7% total coverage (the third-party ISA layer closes separately at 83.1% bins / 89.3% weighted), using only machine-generated, RTL-cited exclusions under a blocking waiver-integrity gate (the unintentional D-extension elaboration at this configuration adds no functional bins and only makes the code-coverage totals conservative; Section XI). Every checker is proven able to fail by permanent fault-injection guards. The flow surfaced real defects on both sides of the comparison, including an externally corroborated JALR LSB ISA deviation and a reset-relay X window found by restoring a silenced assertion. The work is positioned against concurrent RTL GPU fuzzing on the same DUT: complementary—bug discovery by fuzzing versus coverage-quantified sign-off.**

***Index Terms*—UVM, RISC-V, GPGPU, SIMT, lockstep co-simulation, functional coverage, open-source hardware**

I. Introduction

Open-source silicon has an industrial verification problem. The RISC-V ecosystem has mature verification assets for *scalar* cores—lockstep environments, constrained-random generators, ISA coverage libraries [4]–[7], [9]—and sign-off-grade UVM methodology exists for open SoC silicon [20]—but the most complex open processor class, the GPGPU, has none of this. Vortex, the leading open RISC-V GPGPU [1], [2], ships with a directed-kernel regression, no reference-model checking at the RTL level, and no functional-coverage model. Concurrently, FuzzGPU [3] demonstrated that fuzzers can find real bugs in exactly this design space (20 previously unknown bugs, 10 CVEs, across Vortex and Ventus), converting the verification gap into an exposure. Our contributions:

1) **A complete, configurable UVM [10] environment for a SIMT GPGPU** (Section III): role-inverted agents (DUT = AXI master, agents = reactive slaves), a DPI-C–integrated golden model rebuilt per configuration, a passive RVVI-style observability layer, and a bidirectional end-state scoreboard—any topology from plusargs with elaboration-time parameter self-checks.
2) **Per-instruction lockstep for SIMT** (Section V): five alignment rules that make CPU-style lockstep sound for a SIMT machine, validated lane-exact at five configurations (six programs) spanning a $2 \times 2 \times 3 \times 2$ axis space.
3) **A sound two-pass load-value feed** (Section VI) that makes racy, fenceless multi-core programs instruction-granularity verifiable, with non-vacuity proven by injection and a characterized boundary at asynchronous interrupt timing.
4) **A three-layer coverage model with honest closure** (Section VIII): the first published SIMT functional-coverage layer for RTL GPU verification (divergence depth crossed with IPDOM reconvergence, bank conflicts, coalescing classes), closed at 98.1%/94.7% with only machine-generated, RTL-cited exclusions under a blocking waiver-integrity gate.
5) **An evidence-classified findings register** (Sections IX–X): RTL and reference-model defects, each with `file:line` evidence and a disposition—including a reference-model fetch bug found *by* the lockstep itself.

II. Background and Related Work

A. DUT and reference models

Vortex [1], [2] is an open-source RISC-V GPGPU organized as clusters → sockets → cores → warps → threads, with per-core caches, optional shared L2/L3, an immediate-post-dominator (IPDOM) reconvergence stack for divergence, and hardware barriers. The verified build is RV32IMAF at RTL pin `7a52ee5`, primary configuration 1 cluster / 1 core / 4 warps / 4 threads ("1CL"), simulated on QuestaSim. Vortex has *no trap architecture*—no illegal-instruction, misaligned-address,

or CSR exceptions—a property with verification consequences (Section IX).

Vortex ships SimX, a functional (non-timing) simulator maintained with the RTL; it steps per instruction and returns full SIMT architectural state—structurally an RVVI-shaped golden model [6]. Spike [8] cannot execute the SIMT instructions, so SimX is the primary golden model and Spike a *secondary, independent* base-ISA cross-check: on one linked ELF, Spike, SimX and the DUT retire exactly 11,076 writebacks and agree on every PC, destination and value—zero mismatches—with non-vacuity proven by injecting a fault at one record. The audit's scope is bounded by construction: Spike is scalar, so it covers warp 0/lane 0 base-ISA execution only. **The SIMT axis has no independent reference**—so the golden model must itself be treated as a verification target (Section X).

B. Verification and fuzzing prior art

Step-and-compare verification is the standard of care for RISC-V CPUs: core-v-verif [5] established the open-source pattern with ImperasDV [9] over RVVI [6]; riscv-dv [7] supplies constrained-random programs; DiffTest [4] industrialized per-instruction comparison over DPI-C. All target scalar CPUs; none addresses SIMT retirements, warp divergence, or a bus-master GPU DUT.

At the kernel level, GPUVerify [13] proves race- and divergence-freedom of GPU programs and GKLEE [14] concolically tests them, explicitly targeting divergent warps, non-coalesced accesses, and bank conflicts—but on kernel source, not RTL; learned GPU-program generation has precedent in compiler fuzzing [15], and formal memory-consistency verification reaches the RTL level for CPU models [18] and GPU memory models [19].

At the hardware level, FuzzGPU [3] (USENIX Security '26) is the first RTL GPU fuzzer: SIMT-aware program generation, trace-driven differential testing against ISA simulators with per-warp state on Verilator models of Vortex and Ventus, 20 bugs and 10 CVEs. Its coverage is structural (line/toggle/expression), its campaigns time-boxed, and it carries no functional-coverage model, no checker qualification, and no sign-off discipline—by design: it is a bug-discovery engine. Our work is the complementary half: a UVM *methodology* whose claims are coverage-quantified, whose checkers are proven non-vacuous, and whose exclusions are machine-generated and gate-checked. The flows cross-validate: the JALR LSB deviation we report as R1 was independently found by FuzzGPU (S2, upstream PR #339, CWE-682)—in the instruction-set-simulator component of the same project; their RTL findings (X1–X3) are disjoint from ours, consistent with the different pins and oracles involved. Mutation-based checker qualification is established for CPUs and SoCs [16], and systematic fault injection has been brought to GPUs for reliability studies [17]; we apply the qualification discipline end-to-end to a SIMT lockstep flow. Table I summarizes the positioning.

TABLE I
POSITIONING AGAINST REFERENCE-MODEL VERIFICATION AND GPU TESTING FLOWS. "SIMT SEMANTICS" COVERS PER-LANE MASKED COMPARE AND SPLIT-RETIRE AGGREGATION; "CHECKER QUALIFICATION" MEANS FAULT-INJECTION GUARDS; "WAIVER DISCIPLINE" MEANS MACHINE-GENERATED, GATE-CHECKED EXCLUSIONS. ISA COVERAGE FOR OUR L1 LAYER IS PROVIDED BY THE THIRD-PARTY RISCVISACOV LIBRARY; THE SIMT MICROARCHITECTURE MODEL IS THIS WORK'S CONTRIBUTION.

Property	[5], [9]	[4]	[3]	This work
Verification target	scalar CPU	scalar CPU	RTL GPGPU	RTL GPGPU
Per-instruction compare	✓	✓	✓	✓
SIMT semantics	—	—	partial	✓
Racy-program two-pass feed	—	—	—	✓
ISA functional coverage	✓	—	—	✓
SIMT coverage model	—	—	—	✓
Checker qualification	—	—	—	✓
Waiver discipline	—	—	—	✓
Verdict taxonomy	—	—	—	✓
Framing	sign-off	agile DV	bug discovery	sign-off

III. VERIFICATION ENVIRONMENT

Fig. 1 shows the environment.

A. Role-inverted agents

A GPGPU executes programs; it is not driven by them. Five agents surround the DUT: *host* and *device-configuration-register (DCR)* agents (active) drive the launch protocol; the *AXI* agent is an active *responder*—a reactive slave backed by a memory model servicing the DUT's fetch and data traffic; a *memory* agent covers the non-AXI configuration of the same model; a passive *status* agent observes busy/completion state and retire counts. Stimulus is a *program* (compiled ELF); the test class controls only how it is launched, configured, and perturbed.

B. Golden model and probes

SimX is linked via DPI-C (`simx_init`/`load`/`dcr_write`/`run` plus per-retirement record export); RTL and SimX are rebuilt per configuration so DUT and reference always agree on topology. Golden-model crashes are trapped and mapped to a sentinel exit code, so a reference failure is *classified*, never a silent pass. All white-box visibility is bound and passive: a *commit probe* on every core's retire arbiter capturing $\{uuid, wid, PC, rd, wb, tmask, \textit{per-lane data}\}$; an *LSU writeback probe* recovering true per-lane load values; an RVVI-style monitor; and microarchitectural probes (scheduler, divergence/reconvergence, local-memory bank, coalescer, cache, register-hazard, commit-beat) feeding the SIMT coverage layer. Lockstep capture is plusarg-gated; with the gate off, runs are byte-identical to the plain environment.

C. Checkers, configurability, protocol

Two independent checkers render verdicts: (1) a **bidirectional end-state scoreboard**—every word the DUT wrote compared byte-exact against SimX (forward, up to 196,868 words in one test) *and* every word SimX wrote that the DUT never wrote (reverse, catching dropped stores), with per-byte

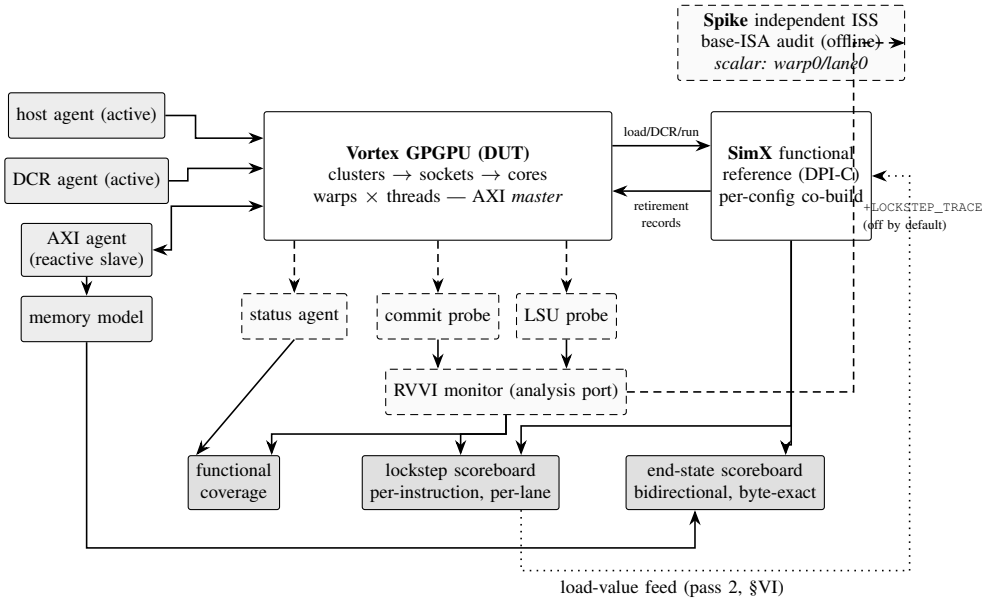

Fig. 1. Environment architecture. Solid gray: active UVM agents; dashed: passive probes and offline tools; dark: verdict-rendering scoreboards and coverage. SimX is the *primary* golden model (live over DPI-C, models SIMT, not independent); Spike is a *secondary, independent* scalar ISS (offline, base-ISA subset only). Neither substitutes for the other.

validity masking and bounded FP tolerance; (2) the **lockstep scoreboard** (Section V). An assertion gate fails any run with a firing RTL assertion, excludes it from coverage merging, and is kept honest by a negative test that must fail through the assertion path alone. One compiled environment runs any topology from plusargs (`+CLUSTERS`/`+CORES`/`+WARPS`/`+THREADS`); elaboration-time assertions compare every UVM parameter against its DUT counterpart. An AXI4 SVA layer (∼15–18 properties plus 11 stability assertions) checks burst legality and stability under backpressure; plusarg-gated *throttle/flood* stress modes and an *error-injection* mode (`+AXI_INJECT_ERR`) drive them.

IV. Verdicts and Non-Vacuity

Every run renders PASS, FAIL, UNVERIFIABLE, or END-STATE-VERIFIED with an instruction-granularity residual. UNVERIFIABLE is first-class: a run the golden model cannot judge is classified with root cause—never force-compared, never counted as a pass. Verdicts are only as good as their ability to go red: every checker is guarded by a permanent fault-injection regression test—wrong-value (`+INJECT_FAULT`) and dropped-store (`+DROP_STORE`) for the end-state scoreboard (both historically detect at the exact address `0x800075d8`); one-bit retirement corruption (`+LOCKSTEP_INJECT`) for the lockstep comparator; and a RAL mirror corruption (`+DCR_RAL_INJECT`) for the register checker. All must stay red on injection after any checker change—the mutation-qualification discipline [16] applied end-to-end. A nearly-accepted *vacuous* gate pass (negative tests reporting PASS when their injection plusargs were absent) was caught and codified: always verify the injection message, never the verdict. This discipline is what allows the coverage and pass-rate numbers below to be taken at face value.

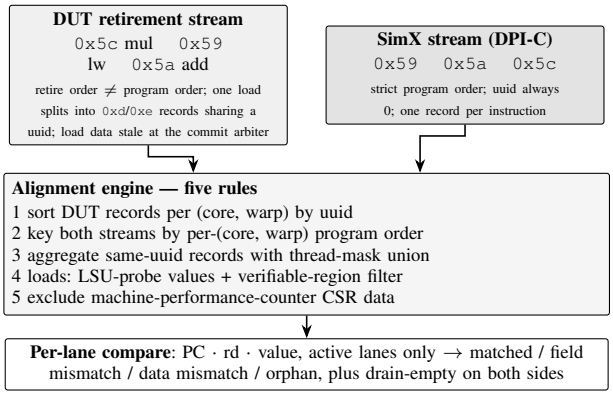

Fig. 2. The lockstep alignment pipeline. The DUT stream (left) carries the SIMT quirks that a scalar-CPU RVVI flow never meets; the reference stream (right) is strictly ordered. Each rule removes exactly one class of *false* mismatch: sorting fixes out-of-order retirements (Rule 1), program-order keying bridges the reference's null uuids (Rule 2), mask-union aggregation reassembles split loads (Rule 3), the LSU probe with region filter restores sound load comparison (Rule 4), and volatile-CSR exclusion removes timing-only divergences (Rule 5). After alignment, any remaining difference outside the three declared exception classes is a real defect.

V. Per-Instruction Lockstep for SIMT

The DUT probe stream and the SimX per-retirement stream meet in a scoreboard that aligns both per (core, warp) and compares each retirement per active lane (PC, destination, value) with a four-way outcome taxonomy and drain-empty checks (Fig. 2). Five rules, each forced by simulation evidence and transferable to any SIMT lockstep, make this sound:

1) Retire order is not program order: The commit arbiter retires whichever unit is ready (`VX_commit.sv:56-71`); an earlier-issued instruction was observed retiring after two later ones. *Align by the per-warp issue counter (uuid), sorted—never by position or cycle.*

2) The golden model's uuid is not a cross-key: SimX leaves its uuid at zero. *Align by per-(core, warp) program order; the DUT uuid's high bits encode* $(CORE\_ID \ll NW\_BITS) + wid$ *(`VX_uuid_gen.sv:40`), supplying attribution with no RTL change.*

3) One instruction is not one record: A single load appears as several records sharing one uuid with partial, overlapping masks (observed `0xd` then `0xe`) as the LSU commits lanes on response arrival. *Aggregate DUT records by uuid with thread-mask union; never by packet flags.*

4) Load data needs its own tap and soundness filter: The commit arbiter's data field is stale for loads; a dedicated LSU probe recovers per-lane values, but a naive compare is *unsound* (429 false mismatches on a known-good kernel from uninitialized regions). *Compare a load lane only when the reference address is in the verifiable region and the golden value is not initialization poison; defer the rest to end-state.* With the filter, the load compare is exact (74 compared, 113 filtered, 0 false mismatches).

5) Performance-counter CSRs diverge by definition: `mcycle`/`minstret`/`mhpmcounter`$\star$ differ between a timing-accurate DUT and a functional model (observed off-by-one). *Flag the MPM CSR range volatile; exclude their data, still check PC and destination.*

Beyond the five rules, the comparator declares three bounded, logged exception classes: the load-value deferral of Rule 4, the volatile-CSR exclusion of Rule 5, and an *fsqrt-only tolerance*. The DUT's `fsqrt.s` is one ULP off correctly-rounded IEEE 754 (Section IX), so an `fsqrt` writeback is tolerated iff the two values are same-sign, finite, and within one ULP (`fp32_within_ulp`); every toleranced comparison is tallied (`n_fp_ulp_tol`)—logged, never silent. All other floating-point operations compare bit-exact, and an `fsqrt` beyond one ULP, or a NaN/Inf mismatch, still fails. A two-pass reconvergence feed (armed with the load feed on `+LOCKSTEP_LOADFEED`) then forces the accepted DUT square-root result into the reference's FP register file and re-checks every downstream operation bit-exactly, so the accepted deviation cannot cascade into false errors; the 13-class FPU kernels verify to residual zero under this handling (1,675 matched retirements). Each rule also carries a DUT precondition an adopter must re-derive: Rule 1 a unique per-warp issue counter that does not wrap within a run; Rule 2 a stable (core, warp) attribution key; Rule 3 records that may split per lane but share an identifier; Rule 4 a tap where true load values are observable; Rule 5 knowledge of the model-divergent CSR ranges.

The lockstep was validated lane-exact—0 mismatches, 0 orphans—across a configuration matrix spanning $NCL \in \{1,2\}$, $NC \in \{1,2\}$, $NW \in \{2,4,8\}$, $NT \in \{2,4\}$ (Table II), including nested-divergence kernels (3v1 $\rightarrow$ 2v1 $\rightarrow$ 1v1) exercising mask-union aggregation through divergence and reconvergence with no comparator changes.

TABLE II
LOCKSTEP CONFIGURATION MATRIX (ALL LANE-EXACT: 0 MISMATCHES, 0 ORPHANS). TALLIES ARE RECORDED IN THE VERIFICATION DOCUMENTATION; PER-RUN RAW LOGS WERE NOT RETAINED.

Configuration	Matched writebacks
1CL/1C/4W/4T (vector-add)	1,035 / 1,035
1CL/1C/4W/4T (nested divergence)	2,668 / 2,668
1CL/2C/4W/4T	1,801 / 1,801
2CL/2C/4W/4T	3,333 / 3,333
1CL/1C/2W/2T	855 / 855
1CL/1C/8W/4T	1,423 / 1,423

VI. VERIFYING RACY PROGRAMS: TWO-PASS LOAD-VALUE FEED

Fenceless multi-core programs have no architecturally defined result on a weakly coherent machine, so per-instruction comparison diverges legitimately: the two models resolve the same race differently. Rather than waive such programs, we make them verifiable. Pass 1 records the DUT's per-lane loaded values at divergent in-region load sites, keyed by (core, warp, PC, occurrence); pass 2 re-runs the reference following those values, so the models share the race outcome and the *residual*, not the feed, is the verdict: a difference not explained by a fed load stays a hard error. The method rests on an assumption we state rather than discharge in general—that a divergent load is a genuine race resolution, not a DUT load-data defect. For the flagship case this assumption was confirmed by trace analysis: the first-divergence load reads a word still holding its pristine ELF initialization value (nothing had overwritten it before the DUT's read), in a single-hart program running on four shared-memory cores—a genuine cross-core ordering race. There, 20 racy loads cascading into 138 divergent retirements collapse to a residual of **zero over 5,432 retirements**, upgrading the verdict from UNVERIFIABLE to *verified modulo the fed race resolutions*. The boundary is stated, not hidden: for an *interrupt* test the feed collapses 116 divergences to 7 but not 0, because the two models take the interrupt at different instruction boundaries; re-keying the feed leaves the residual identical, proving it is not an alignment artifact. Interrupt-timing divergence therefore remains instruction-granularity unverifiable, while end-state still passes. *Two-pass replay is a fixed point for data-only divergence; asynchronous-input timing requires a step-follower reference.*

VII. STIMULUS

Directed kernels ($\sim$30, all byte-exact against the reference): arithmetic/memory baselines; all 13 FPU classes; tensor WMMA including multi-warp launches; nested-divergence towers; barriers under partial masks; `wspawn`/`tmc` sweeps; MSHR and memory-pressure stress; a 232 KB-text kernel; a 256 KB store stress; VOTE/SHFL. **Constrained-random**: a riscv-dv [7] pipeline targeting rv32im with GPU-specific

post-processing; two profiles are excluded as *unimplementable* (unaligned access, no misaligned support; illegal instruction, no trap architecture), and jump-stream generation is sanitized to keep riscv-dv's deliberate JALR-LSB probing away from R1—the deviation is reported, not hidden. **simtgen**: an original random C-kernel generator targeting the microarchitectural axes scalar generators cannot reach, with knobs derived from the DUT's own configuration space (warp/thread counts, coalescer window, local-memory bank count, IPDOM stack depth), informed by the generation-axis taxonomy published by FuzzGPU [3] but implemented against our own checking stack; no third-party code is used. `simtgen` implements four axes: divergence (structured branch trees with controlled split depths), memory (stride/scatter/bank-hostile patterns), barrier topology (single-warp barriers, deadlock-safe by construction), and VOTE/SHFL (all eight intrinsics per program). The barrier and VOTE/SHFL axes closed no new coverage—their targets (`cp_vote_shfl_op` 8/8, `cross_sfu_threads`) were already fully covered by dedicated directed kernels; their contribution is generator completeness and seed-diversity robustness evidence, the same category of result as the riscv-dv seed farm below. A ten-seed sweep across nine riscv-dv profiles (90 additional distinct programs, content-hash verified, all passing) produced *no measurable functional-coverage gain* (every category bit-identical except toggle $+0.06\%$)—a robustness result reported as such: coverage saturates inside the scalar generator's reach, and the remaining work is stimulus of a different *kind*.

VIII. Coverage Methodology and Results

A. Three layers, never merged

L1 — ISA (third-party riscvISACOV): 80 active covergroups over RV32I/F/M/Zicsr/Zifencei. Sampling *every active lane as a hart* (4,581 samples) was proven to cover exactly the same bin set as lane-0-only (1,677)—empirical proof that ISA coverage is blind to the SIMT axis, which is why L2 exists. A closed gap-hunt leaves 78/80 covergroups real and 429/516 target bins hit (83.14%; 89.28% weighted) after register-index exclusions. **L2 — SIMT microarchitecture** (this work's contribution): probe-sampled covergroups for warp divergence crossed with IPDOM depth (`divergence_cg`), reconvergence, barrier scope/events, `tmc`/`wspawn`, scratchpad bank conflicts (`lmem_bank_cg`), coalescing classes (`coalesce_cg`), cache events, register hazards, and commit-beat shapes. **L3 — system/protocol**: SVA assertion coverage, AXI transaction space, DCR/host launch spaces. Layer databases are never blended.

B. Closure discipline

(1) *Exclusions are structural, RTL-cited, machine-generated*: each waiver carries a `file:line` citation (e.g., write-response stability is unreachable because the adapter hardwires `m_axi_bready=1'b1`, `VX_axi_adapter.sv:313`), and a blocking *hits-invariant* merge gate verifies every waived bin had zero hits unwaived—on first execution it caught two real waiver defects, one latent for weeks. (2) *Reachable-but-unhit stays red*: never waived. (3) *Ceilings are root-caused*: the toggle plateau was traced per sub-module to a read-only icache data port (51,340 bins, 55.7% sub-module toggle; 26.4% of the entire toggle gap from one subtree, `VX_socket.sv:106`) and constant address bits; an adversarial maximum-entropy kernel moved aggregate toggle $+0.02\%$, establishing the ceiling as structural. Banks are per-configuration: cross-topology UCDB merging was shown invalid (instance-set inflation).

TABLE III
COVERAGE BANKS (QUESTASIM; PER-CONFIGURATION, NEVER BLENDED). ALL ROWS ARE THE 2026-08-16 *pre-relay-fix* BANKS; THE RESET-FIXED 1CL BANK (51 RUNS, 2026-08-18) TOTALS THE SAME 94.7% WITH BRANCH 94.53% AND TOGGLE 83.34%—INDICATIVE ONLY, TWO VARIABLES CHANGED (§VIII-E). A RELAY-FIXED 2CL BANK TAKEN AFTER THIS TABLE WAS FROZEN (110 RUNS, 0 FAILURES, 93.4% COVERGROUP BINS / 94.3% TOTAL, ON THE GROWN POST-REMEDIATION COVERAGE MODEL) RESTORES CROSS-CONFIGURATION COMPARABILITY; THE PRE-FIX COLUMNS ARE RETAINED FOR PROVENANCE.

Metric	1CL/1C/4W/4T	2CL/2C/4W/4T
Covergroup bins (raw)	370/377 = 98.1%	989/1032 = 95.8%
Covergroup (weighted)	99.8%	99.5%
Statement	98.1%	98.3%
Branch	95.1%	95.7%
Condition	90.4%	88.8%
Toggle	82.8%	80.5%
Assertion	96.9%	98.9%
Directive	100.0%	100.0%
Total	**94.7%**	**94.6%**
Runs passing	50/50[‡]	50/50
Coverage instances	2 256	8 275

[‡] *Simulation runs*, not programs: 49 distinct programs, one run twice under two bus modes.
A third bank with shared L2/L3 enabled totals 93.2% (51 runs, all passing), also on the pre-fix design.

C. Results

Table III summarizes the banked configurations.

D. SIMT-layer closures and honesty notes

Before simtgen, three SIMT targets were open: depth-4 IPDOM divergence (`cp_split_depth` 3/4—depth 3 unhit across 50 seeds), the bank-conflict bin (`lmem_bank_cg` 0/3, never sampled in 70,498+ cycles), and one apparent coalescing gap. Deliberate simtgen programs closed the first two (Fig. 3): depth 4/4 and real bank conflicts surfaced; both closures are folded into a suite bank (2026-09-09). Three attribution notes travel with this result, each established by experiment: (a) the bank-conflict "gap" was in fact a *coverage-model defect*: the coverpoint was defined over *accepted* requests, but the local-memory crossbar admits exactly one winner per bank per cycle—"two accepted requests on one bank" was structurally impossible, and deliberate bank-hostile stimulus proved it (zero conflicts); reclassifying onto *offered* requests made the bin reachable and honest (169 conflict hits), so the closure credit is shared between the probe fix and the

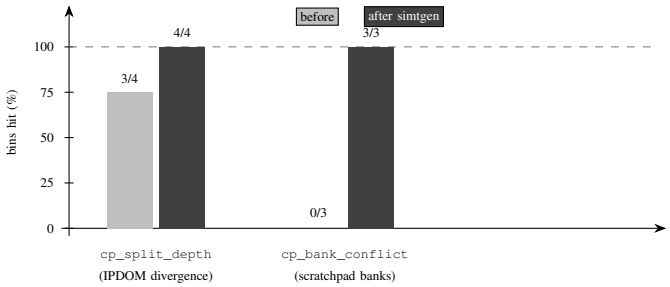

Fig. 3. Effect of axis-directed generation on the two SIMT targets it genuinely closed (bins hit, as a percentage of the coverpoint's bin count). Before simtgen (light bars): depth-4 IPDOM divergence was unreachable by any of 50 random-program seeds (3/4 bins); bank conflicts never occurred in 70,498+ sampled cycles (0/3—itself root-caused to a coverpoint-definition defect, since the crossbar admits one winner per bank per cycle). Deliberate divergence-tower and bank-hostile kernels (dark bars) closed both to 100%, now folded into a suite bank (2026-09-09). A third target previously reported here (coalescing classes) was re-examined and found already covered by the existing coalescer probe and kernels before simtgen existed—an attribution correction to our earlier isolated-merge reporting.

generator; (b) the coalescing classes, which an earlier isolated-merge report attributed to simtgen, were re-examined and found already covered by the existing coalescer probe and kernels—an attribution correction we report rather than quietly keep; (c) the barrier and VOTE/SHFL generator axes closed nothing because their targets were already fully covered by directed kernels (above). A coverage definition must itself be validated against the RTL's structure; the stimulus gap and the taxonomy defect were disentangled by experiment. The marginal picture per generator kind: ninety additional riscv-dv programs closed zero bins (the seed-farm result), while 57 simtgen programs (divergence and memory axes) closed the divergence and bank-conflict targets—stimulus *kind*, not volume, moves this coverage model.

E. Provenance of the DUT

The design is pinned at a specific revision *with eighteen locally modified files in the synthesizable RTL tree* (mostly simulator-compatibility fixes; the environment is new code). We disclose this because a verification result is a statement about a specific artifact. One modification's history is instructive: library counter assertions fired during bring-up and were guarded off (skip on unknown inputs) so the environment could run—for months. Restoring the original assertion and root-causing it exposed R10 (Section IX-A); after the fix, the assertions pass with no guard and the local modification was retired in favor of the unmodified upstream file. Fixing the defect *reduced* branch coverage (95.1%→94.5%): modules behind a relay had been executing normal-operation paths during an unknown-reset cycle, counted as covered branches. No coverage metric can report that some of its coverage came from a state that cannot legitimately arise; we report the lower figure. (Two variables changed; the delta is indicative.)

TABLE IV
RTL FINDINGS. THE R1 SPEC DEVIATION WAS INDEPENDENTLY FOUND BY FUZZGPU [3] IN THE ISS COMPONENT (S2, PR #339); THEIR RTL FINDINGS (X1–X3) ARE DISJOINT FROM OURS; R2 WAS INDEPENDENTLY FIXED UPSTREAM.

ID	Finding	Class	Disposition
R1	JALR LSB not cleared; odd PC propagates via AUIPC	Bug (ISA)	needs RTL fix; stimulus sanitized
R2	`STALL_TIMEOUT` `1**N≡1`; watchdog never scales	Bug (latent)	fixed; independently fixed upstream
R3	Misaligned access: no trap; silently retargeted/torn	Hazard	per SW contract; gated
R4	Core self-starts; DCRs unreset	Hazard	worked around
R5	Per-warp out-of-order commit	Quirk	handled (§V)
R6	One load → multi records, overlapping masks	Quirk	handled (§V)
R7	uuid encodes flat core+warp id	Quirk	exploited (§V)
R8	Load data unobservable at commit arbiter	Observability	closed (LSU probe)
R9	Fire-and-forget writes; *no AXI error path*	Characteriz.	cited; fault-injected (166)
R10	Reset relay: reset registered in an unreset flop; X for one cycle	Bug (X)	fixed; still upstream

IX. FINDINGS I: RTL

All findings are maintained in an evidence-cited register (one entry each: what was seen, `file:line`/run evidence, class, disposition). Table IV summarizes R1–R10.

A. Unknown reset for one cycle, found by restoring a silenced assertion (R10)

Reset is distributed through a relay that registers it in a flip-flop nothing resets:

```
`PRESERVE_NET reg [R-1:0] reset_r;   // no initial value
always @(posedge clk) reset_r[i] <= reset;
assign reset_o[i] = reset_r[i / F]; // X until first edge
```

Every module behind a relay observes an unknown reset for one cycle; `if (X)` takes the non-reset branch, so reset-held logic executes and assertions evaluate on unknown operands. The counter assertions that first flagged this had been *guarded off for months*; the assertion had been correct the entire time—what was suppressed was the report, not the problem. We replaced the relay with an async-assert/sync-deassert synchronizer: twelve firings become zero with the upstream file unmodified, a 51-run regression passes with assertions armed, and the injection guards were re-confirmed. The defect remains in the current upstream release. Two generalizations: **a checker disabled to make a bench run is a checker whose findings are lost—silently and open-endedly**; and **fixing a defect can reduce a coverage number** (Section VIII-E).

B. JALR does not clear the target LSB (R1)

The RISC-V specification [11] requires JALR to clear the target LSB. Vortex omits the clear (`VX_alu_int.sv:222`), and with no trap architecture cannot raise the prescribed misaligned-target exception. In the debug build (`PC_BITS=XLEN`) the odd bit survives as the *architectural*

PC: fetch silently word-aligns, so execution continues correctly, but `auipc`/`la` inherit the skew, link registers accumulate it across chained jumps ($+1 \rightarrow +3$ observed), and downstream accesses go misaligned (R3). The trigger is spec-legal: riscv-dv deliberately sets the JALR base LSB expecting the clear, so roughly half of generated jumps derail—every profile of a 12-profile suite fired misaligned assertions (30–7,616 per run). The release build is spec-correct *by accident* (PC bits masked). FuzzGPU [3] found the same deviation independently from an entirely different flow (S2, upstream PR #339—filed against the project's instruction-set simulator; their RTL findings X1–X3, at a different pin, are disjoint from ours)—independent corroboration of the *deviation*, though not of its RTL embodiment.

C. Further verified defects

fsqrt.s one ULP off correctly-rounded IEEE 754 [12]: a lockstep lane-0 mismatch (DUT `0x3fef7750` vs. reference `0x3fef7751`) localized to the sqrt unit; the end-state FP tolerance absorbs it—*only* per-instruction comparison surfaces it. **Memory-scheduler timeout misdenominated**: beside R2, a picoseconds-denominated localparam yields $\sim$1,000-cycle watchdogs, flooding L2/L3 runs with 22,187 false alarms while functionally clean. **No AXI error path** (R9): the adapter hard-asserts `OKAY`; injecting `SLVERR`/`DECERR` every 7th transaction produced 166/166 assertion firings and no recovery path—an evidence-based upgrade of a waiver into a measured design limitation.

X. Findings II: Reference Model

A lockstep flow verifies the reference model as much as the DUT. SimX read instructions at the exact byte PC; a computed jump to an odd target (tolerated by the RTL, which word-aligns) made the reference read across an instruction boundary and abort, silently classifying runs unverifiable. The lockstep's divergence report localized it; word-aligning the reference fetch recovered the runs. The *obvious* fix—masking the JALR target in the model—was tried and *reverted with evidence*: 21,968 phantom mismatches, because it fixed the model to the specification rather than the DUT. Further: SimX aborts on unknown input at 56 disassembly-verified sites (35 decode, 21 execute), mostly in the pretty-printer—a run can be unverifiable because the golden model could not *name* an instruction; lockstep upgrades such verdicts to "byte-exact for all 17,664 retirements before abort." A SIMT-divergent CSR access produced 109 genuine mismatches (reference serialized lanes over the per-warp `fcsr`; RTL reads-broadcast/writes lane 0)—the model was corrected; the RTL quirk is documented, not mirrored.

XI. Limitations and Soundness Boundary

- **Asynchronous interrupt timing**: the interrupt test collapses 116→7 but not 0; the residual is keying-independent and genuine (Section VI)—end-state VERIFIED, instruction-granularity residual classified, not forced green.
- **Control-flow-steering races**: when racy loads feed branches, pass-2 replay meets fresh unkeyed races (residual 15, thirteen of them load-class); no fixed point exists—left honestly red.
- **Structural blindness of differential testing**: DUT and reference execute the same binary, so a stimulus fault (e.g., a kernel whose compile-time geometry mismatches the RTL configuration) is invisible—both models wrong identically while every checker passes; equivalence validates only what *differs*; shared inputs need independent assertions. We closed exactly one such input and report the class open.
- **No independent SIMT reference**—structural ceiling, not unfinished task.
- **D-extension elaboration, scoped by inspection**: the primary configuration elaborates the D extension and MISA reports D=1 (`FLEN=64`, confirmed by elaboration probe) although the toolchain builds RV32IMAF and the reference models F only—unintentional-by-documentation as an RTL fact. Its coverage impact is bounded by code inspection: the coverage model's only D-specific bin (float–double conversion) is config-aware-excluded at RV32 (`ignore\_bins`), the ISA layer compiles no D plan, and no stimulus can emit a D operation—so no functional bin is diluted. The elaborated-but-unexecuted D logic sits only in code-coverage denominators, which makes the reported totals *conservative* (they understate an F-only build), never inflated.
- **Per-mnemonic aliasing**: the frozen banks predate discovery that instruction-class encodings alias (`4'b1011` is both a zero-conditional op and `EBREAK`); per-mnemonic claims use only post-remediation measurements.
- **Generality**: all results are one DUT, one simulator, one build; the alignment rules' DUT preconditions are stated in Section V, and transfer beyond them is belief, not demonstration.
- **Scope**: this is front-end RTL functional verification—gate-level, timing, DFT, CDC, and physical sign-off are prerequisites for a different claim entirely and are out of scope.

XII. Conclusion

We presented a UVM environment and closure methodology that brings reference-model verification to a SIMT GPGPU, with the first published SIMT functional-coverage layer for RTL GPU verification. Five alignment rules, each with a stated DUT precondition, make per-instruction lockstep sound for SIMT; a two-pass load feed verifies racy programs at instruction granularity (residual 0 over 5,432 retirements, verified modulo the fed race resolutions) with an honestly characterized boundary; the closure methodology—machine-generated, gate-checked exclusions, root-caused ceilings, residual bins dispositioned rather than waived—carried the primary configuration to 98.1%/94.7%; and every verdict is backed by injection-proven checkers. The checking depth paid for itself on both sides of the comparison—an externally corroborated

ISA deviation, an upstream-fixed watchdog defect, a reset-relay X window found by unsuppressing an assertion, and a reference-model fetch bug found by the lockstep itself. The central lesson—the reference model must be verified alongside the DUT, and the coverage model alongside both—transfers to other open SIMT designs and fills the half of the open-GPU verification problem that fuzzing does not address.

REFERENCES

[1] B. Tine, K. P. Yalamarthy, F. Elsabbagh, and H. Kim, "Vortex: Extending the RISC-V ISA for GPGPU and 3D-graphics," in *Proc. 54th IEEE/ACM Int. Symp. Microarchitecture (MICRO-54)*, 2021, pp. 754–766.

[2] F. Elsabbagh *et al.*, "Vortex: OpenCL compatible RISC-V GPGPU," in *Workshop on Computer Architecture Research with RISC-V (CARRV)*, 2019.

[3] Z. Gao, J. Wang, Q. Zhou, L. Shan, J. Hu, X. Jia, and Z. Lv, "Fuzzing open-source GPU hardware with SIMT program generation," in *Proc. 35th USENIX Security Symposium (USENIX Security '26)*, Baltimore, MD, Aug. 2026, pp. 2465–2484.

[4] Y. Xu *et al.*, "Towards developing high-performance RISC-V processors using agile methodology," in *Proc. 55th IEEE/ACM Int. Symp. Microarchitecture (MICRO-55)*, 2022.

[5] OpenHW Group, "core-v-verif: functional verification project for the CORE-V family of RISC-V cores," [Online]. Available: https://github.com/openhwgroup/core-v-verif

[6] OpenHW Group / Imperas, "RVVI: RISC-V Verification Interface," [Online]. Available: https://github.com/riscv-verification/RVVI

[7] CHIPS Alliance, "riscv-dv: SV/UVM based instruction generator for RISC-V processor verification," [Online]. Available: https://github.com/chipsalliance/riscv-dv

[8] RISC-V International, "Spike, the RISC-V ISA simulator," [Online]. Available: https://github.com/riscv-software-src/riscv-isa-sim

[9] Imperas Software, "ImperasDV: processor verification IP with lock-step compare," [Online]. Available: https://imperas.com/imperasdv

[10] *IEEE Standard for Universal Verification Methodology Language Reference Manual*, IEEE Std 1800.2-2020, 2020.

[11] A. Waterman and K. Asanović, Eds., *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, RISC-V Foundation, 2019.

[12] *IEEE Standard for Floating-Point Arithmetic*, IEEE Std 754-2019, 2019.

[13] A. Betts, N. Chong, A. F. Donaldson, S. Qadeer, and P. Thomson, "The design and implementation of a verification technique for GPU kernels," *ACM Trans. Program. Lang. Syst.*, vol. 37, no. 3, pp. 10:1–10:49, 2015.

[14] G. Li, P. Gopalakrishnan, I. Ghosh, S. P. Rajan, and G. Gopalakrishnan, "GKLEE: Concolic verification and test generation for GPUs," in *Proc. ACM SIGPLAN Symp. Principles and Practice of Parallel Programming (PPoPP)*, 2012, pp. 215–224.

[15] C. Cummins, P. Petoumenos, H. Leather, and S. Fursin, "Compiler fuzzing through deep learning," in *Proc. ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA)*, 2018, pp. 95–105.

[16] L. Di Guglielmo, F. Fummi, and G. Pravadelli, "Vacuity analysis for property qualification by mutation of checkers," in *Proc. Design, Automation & Test in Europe (DATE)*, 2010, pp. 262–267.

[17] T.-T. Tsai, S. K. S. Hari, M. Sullivan, O. Villa, M. B. Glick, and S. W. Keckler, "NVBitFI: Dynamic fault injection methodology for GPU applications," in *Proc. IEEE/IFIP Int. Conf. Dependable Systems and Networks (DSN)*, 2021, pp. 64–76.

[18] Y. A. Manerkar, D. Lustig, M. Martonosi, and M. Pellauer, "RTLCheck: A property checking approach for custom memory consistency models," in *Proc. 50th IEEE/ACM Int. Symp. Microarchitecture (MICRO-50)*, 2017, pp. 713–725.

[19] D. Lustig, S. Sahasrabuddhe, and M. Giroux, "A formal analysis of the NVIDIA PTX memory model," in *Proc. Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019, pp. 253–266.

[20] lowRISC, "OpenTitan: open source silicon root of trust," [Online]. Available: https://github.com/lowRISC/opentitan